UrbanFlow CDMX

Inventario Completo del Proyecto

Qué se hizo · Con qué herramientas · En qué orden

Carlos Armando López Encino

Diplomado en Ciencia de Datos
FES Acatlán · UNAM

Proyecto Integrador Final

Abril 2026

Propósito de este documento

Este documento es un **inventario exhaustivo** de todo lo que se construyó durante el proyecto integrador del Diplomado en Ciencia de Datos: herramientas, bibliotecas, fuentes de datos, decisiones arquitectónicas, entregables y recursos consultados. Está organizado **cronológicamente por fase de trabajo** para que refleje el orden real en que fueron apareciendo las piezas del proyecto.

No es un documento de marketing ni un pitch; es un registro técnico interno del esfuerzo realizado, útil para cualquier persona que quiera reproducir el trabajo, continuar el desarrollo, o simplemente entender el alcance del proyecto.

Nota: las fechas exactas de cada fase están marcadas con `[POR CONFIRMAR]` en los lugares donde el autor debe completarlas antes de entregar este documento al profesor.

1 Fase 0 — Concepción y planteamiento [POR CONFIRMAR: nov–dic 2025]

1.1 Definición del problema

- Identificación del tráfico en la ZMVM como problema no resuelto por sistemas de navegación convencionales
- Revisión del TomTom Traffic Index 2024 y confirmación de que CDMX es la ciudad #1 en congestión mundial
- Definición del gap: Google Maps/Waze entregan estimaciones puntuales; ningún sistema entrega bandas de incertidumbre específicas del contexto mexicano

1.2 Elección del enfoque metodológico

- Decisión de usar **cadenas de Markov de tiempo discreto** como modelo de dinámica de tráfico
- Decisión de complementar con **simulación Monte Carlo** para cuantificar incertidumbre
- Justificación académica: combina dos temas del diplomado (procesos estocásticos + métodos de simulación) en un problema realista

1.3 Fuentes consultadas en la fase inicial

- TomTom Traffic Index 2024 — `https://www.tomtom.com/traffic-index/`
- INEGI — Encuesta Origen-Destino en Hogares ZMVM 2017
- DataMéxico (Secretaría de Economía) — perfil económico de Valle de México
- Mordor Intelligence — Mexico Last Mile Delivery Market Report 2025–2030
- Norris, J. R. (1997). *Markov Chains*. Cambridge University Press
- Rubinstein & Kroese (2016). *Simulation and the Monte Carlo Method*. Wiley

2 Fase 1 — Arquitectura del sistema [POR CONFIRMAR: dic 2025]

2.1 Decisiones arquitectónicas principales

- Separación clara entre **motor estocástico** (Python puro vectorizado) y **capa conversacional** (agente LLM)
- Estructura de directorios estilo `src/`:

```
UrbanFlow_CDMX/
|-- src/
|   |-- simulation/       (markov_chain.py, monte_carlo.py)
|   |-- data_sources/     (tomtom_client.py, weather_client.py, c5_client.py)
|   |-- agent/            (agent.py, perturbaciones.py, tools.py)
|   +-- core/             (schemas.py Pydantic)
|-- app/
|   +-- streamlit_app.py
|-- tests/                (pytest)
|-- notebooks/            (EDA)
|-- data/                 (caches parquet)
+-- .env
```

- Elección de **Anthropic API** sobre OpenAI por mejor razonamiento sobre tablas probabilísticas
- Elección de **Streamlit** sobre Gradio por integración más madura con Folium (mapas interactivos)

2.2 Stack tecnológico inicial

Capa	Herramienta	Rol
Cómputo numérico	NumPy 1.26	Álgebra lineal vectorizada (Markov, Monte Carlo)
Datos tabulares	Pandas 2.2	DataFrames de tráfico, clima, rutas de validación
Geoespacial	GeoPandas	Proyección EPSG:6372 para cálculos de distancia
Mapas	Folium	Renderizado interactivo de rutas sobre tiles CDMX
Validación esquemas	Pydantic 2.x	Structured outputs y contratos tipados
Frontend	Streamlit 1.56	Dashboard interactivo con mapa + chat
Agente LLM	`anthropic` SDK	Cliente Python para Claude API
HTTP asíncrono	`httpx`	Llamadas concurrentes a APIs externas
Testing	pytest	443 tests automatizados
Gestión deps	`pip-tools`	Lockfile reproducible en `requirements.txt`
Entorno	`python-dotenv`	Carga de API keys desde `.env`

3 Fase 2 — Fuentes de datos e ingesta [POR CONFIRMAR: ene 2026]

3.1 Cinco fuentes de datos integradas

Fuente	Tipo	Uso en el sistema
TomTom Traffic Stats API	REST, tiempo real	Velocidad actual y de flujo libre por coordenada. Inferencia del estado inicial s_0 de la cadena de Markov. Cuota gratuita: 2,500 req/día.
TomTom Routing API	REST, por consulta	Distancia real por carretera y geometría de ruta (polilínea). *Ground truth* para validación empírica.
OpenWeatherMap API	REST, cada 10 min	Temperatura, precipitación, condición climática. Factor ϕ sobre velocidades. Cuota gratuita: 1,000 req/día.
C5 CDMX Histórico	CSV desde portal CKAN	Serie histórica de incidentes viales 2018–2024 (~1M registros). Calibración de la matriz de transición $\hat{P}$ y densidad horaria.
Google Maps Distance Matrix	REST, por consulta	Validación cruzada del ETA, geocodificación de direcciones del usuario.

3.2 Portal de datos abiertos CDMX

- https://datos.cdmx.gob.mx/dataset/incidentes-viales-c5 — dataset C5 principal
- CKAN API: https://datos.cdmx.gob.mx/api/3/action/package_show

3.3 Módulos de cliente implementados

- `src/data_sources/tomtom_client.py` — wrapper de las dos APIs de TomTom
- `src/data_sources/weather_client.py` — cliente OpenWeatherMap con mapeo de condiciones climáticas a factor ϕ
- `src/data_sources/c5_client.py` — descarga y limpieza del CSV histórico de C5 CDMX
- `src/data_sources/gmaps_client.py` — geocodificación y Distance Matrix

3.4 Retos encontrados y cómo se resolvieron

- **Cuotas API limitadas**: Se implementó un sistema de caché local con `requests-cache` para evitar llamadas repetidas durante desarrollo
- **Indisponibilidad ocasional de TomTom Routing**: Se implementó el fallback empírico $d_{\text{carretera}} \approx d_{\text{haversine}} \times 1.4$, calibrado sobre 500 pares O-D aleatorios
- **Nombres de columnas distintos por año en C5**: Se creó un mapeo `MAPEO_COLUMNAS` que normaliza nombres a un esquema único

4 Fase 3 — Análisis exploratorio de datos [POR CONFIRMAR: ene–feb 2026]

4.1 Notebook EDA v3

Archivo: `notebooks/EDA_UrbanFlow_CDMX_v3.ipynb`

- Limpieza y tipado de ~1 millón de registros del C5 CDMX

- Análisis de valores faltantes con umbrales de 5 % y 10 %
- Distribución geográfica (hexbin map) de incidentes sobre el bounding box ZMVM
- Distribución temporal: hora del día, día de la semana, mes, heatmap hora×día
- Identificación de los dos picos diarios: matutino 7–9 h y vespertino 17–20 h
- Top alcaldías por incidentes: Iztapalapa, Cuauhtémoc, Gustavo A. Madero
- Proxy de congestión por densidad horaria normalizada a $[0.35, 1.0]$

4.2 Bibliotecas adicionales de visualización

- `matplotlib` — gráficos base
- `seaborn` — heatmaps y estilos
- `scipy.stats` — distribuciones y tests

5 Fase 4 — Motor estocástico (Markov + Monte Carlo) [POR CONFIRMAR: feb 2026]

5.1 Clase `MarkovTrafficChain`

Archivo: `src/simulation/markov_chain.py`

- Interfaz estilo scikit-learn: `fit()` → `predict_distribution()` → `simulate()`
- Estimación MLE con suavizado de Laplace ($\varepsilon = 10^{-6}$) para evitar probabilidades cero
- Cálculo del estado estacionario π^* via descomposición espectral de $P^\top$
- Soporte para 3 estados: Fluido / Lento / Congestionado
- Matriz calibrada para el corredor Insurgentes Sur en pico AM

5.2 Clase `MonteCarloEngine`

Archivo: `src/simulation/monte_carlo.py`

- Simulación vectorizada sobre N trayectorias (10,000 por defecto)
- Método de la **transformada inversa vectorizada** para muestrear estados de Markov en paralelo
- Muestreo de velocidades por estado usando Normal truncada
- Interpolación lineal sub-paso para precisión de tiempo de llegada
- Cálculo de percentiles P_{10}, P_{50}, P_{90} y métrica derivada IC
- Latencia: <500 ms por consulta en CPU sin GPU

5.3 Retos técnicos

- **Vectorización**: la versión naive con bucles Python tomaba ∼30 segundos por consulta. La vectorización por transformada inversa la redujo a <500 ms (60× mejora)
- **Estabilidad numérica**: se agregó normalización después de cada operación sobre la matriz para evitar derivas acumuladas

5.4 Cobertura de tests

- `tests/test_markov_chain.py` — propiedades de estocasticidad, convergencia estacionaria, predicción en k pasos
- `tests/test_monte_carlo.py` — convergencia del estimador, reproducibilidad con seed, rangos de bandas
- `tests/test_pipeline.py` — integración end-to-end desde ratio de congestión hasta bandas P10/P50/P90
- Total: **443 tests pasando al 100 %**

6 Fase 5 — Catálogo de perturbaciones contextuales [POR CONFIRMAR: feb–mar 2026]

6.1 Motivación

TomTom y OpenWeatherMap no pueden capturar eventos como cierres del Metro, manifestaciones o eventos masivos hasta que ya pasaron. Se construyó un catálogo de perturbaciones que modifica la matriz de Markov *antes* de ejecutar la simulación.

6.2 Formulación matemática

Función de perturbación por factor multiplicativo:

$$p'_{i,2} = \min(f \cdot p_{i,2},\, p_{\max}), \quad p_{\max} = 0.92$$

con renormalización automática de la fila para preservar estocasticidad.

6.3 Catálogo implementado

Archivo: `src/agent/perturbaciones.py`

- **Infraestructura**: Cierre Línea 1 Metro ($f = 1.25$), Cierre Línea 12 Metro ($f = 1.20$)
- **Eventos masivos**: Concierto en Centro Banamex ($f = 1.15$), Partido en Estadio Azteca ($f = 1.22$), Concierto en Foro Sol ($f = 1.20$)
- **Manifestaciones**: Marcha 9 de marzo ($f = 1.30$), Marcha magisterial CNTE ($f = 1.27$)
- **Festivos alto impacto**: 15 de septiembre — Grito ($f = 1.35$), 16 de septiembre — Desfile ($f = 1.30$)
- **Festivos bajo impacto**: Navidad / Año Nuevo ($f = 0.55$)

Total: **11 eventos** calibrados empíricamente contra datos históricos del C5 CDMX.

7 Fase 6 — Agente conversacional VialAI [POR CONFIRMAR: mar 2026]

7.1 Arquitectura del agente

- Patrón *tool use loop* estándar de la API de Anthropic
- Modelo: `claude-sonnet-4-5` (o versión disponible en el momento)
- Integración con Streamlit via `st.session_state` para mantener historial

7.2 Structured Outputs con Pydantic

Archivo: `src/core/schemas.py`

- `RespuestaTomTom` — valida respuesta de TomTom Traffic Stats
- `RespuestaClima` — valida respuesta de OpenWeatherMap
- `PrediccionViaje` — resultado estructurado del motor (P10/P50/P90, IC, semáforo)
- `PerturbacionActiva` — evento contextual con tipo, radio, duración

7.3 Tres herramientas expuestas al agente

- `predecir_tiempo_viaje(origen, destino, hora)` → PrediccionViaje
- `consultar_trafico_ahora(zona)` → Estado actual + ratio de congestión
- `verificar_perturbaciones(zona, hora)` → Lista de PerturbacionActiva

7.4 Reto: Bug de carga de `.env`

- Síntoma: `ANTHROPIC_API_KEY` no cargaba cuando Streamlit instanciaba el agente
- Causa: `load_dotenv()` sin argumentos usa *cwd* como referencia, pero Streamlit corre desde un directorio distinto al del módulo
- Solución: ruta absoluta derivada del propio archivo vía `Path(__file__).resolve().parents[2]`

8 Fase 7 — Interfaz Streamlit [POR CONFIRMAR: mar 2026]

8.1 Componentes de la UI

- Mapa interactivo Folium con selección de origen/destino por click
- Widget de clima en tiempo real desde OpenWeatherMap
- Gauge del Índice de Confiabilidad con semáforo verde/amarillo/rojo
- Histograma de la distribución Monte Carlo
- Sidebar con chat de VialAI
- Toggles para activar perturbaciones del catálogo

8.2 Archivo principal

`app/streamlit_app.py`

8.3 Dependencias adicionales

- `streamlit-folium` — integración mapa-Streamlit
- `plotly` — gauges interactivos del IC

9 Fase 8 — Validación empírica [POR CONFIRMAR: mar 2026]

9.1 Dataset de validación

30 rutas representativas muestreadas entre 9 polos de actividad de la ZMVM: Polanco, Santa Fe, Reforma, Roma-Condesa, Insurgentes Sur, Coyoacán, Aeropuerto, Centro Histórico, Naucalpan.

9.2 Métricas reportadas

- **MAPE del P_{50} (VialAI): 11.9 %** — bajo el umbral objetivo del 15 %
- **MAPE del ETA (TomTom Routing): 19.0 %** — baseline competitivo
- **VialAI supera a TomTom por 7.1 puntos porcentuales**
- **Cobertura empírica de la banda P_{10}–P_{90}: 93.3 %** (nominal 80 %)
- **MAE del P_{50}: 4.8 minutos**

9.3 Casos testigo

- **Caso ganado**: Polanco → Aeropuerto (08:15). TomTom ETA: 38 min. VialAI P50: 40 min. Real: 47 min (dentro de P90). TomTom habría subestimado en 9 minutos.
- **Caso perdido**: Coyoacán → Santa Fe (09:30). Real: 95 min debido a manifestación no anunciada. Fuera de la banda P90 de VialAI. Este caso motivó la construcción del catálogo de perturbaciones.

10 Fase 9 — Documentación final [abril 2026]

10.1 Artículo técnico

- **Archivo**: `UrbanFlow_CDMX_Articulo_v2.pdf` (23 páginas)
- Fuente: LaTeX con `tcolorbox`, `TikZ`, `listings`, `microtype`
- 12 secciones: motivación, arquitectura, datos, Markov, Monte Carlo, perturbaciones, agente, validación, estrategia B2B, conclusiones
- 4 `challengebox` documentando retos técnicos reales
- 4 `insightbox` con hallazgos vendibles
- 17 referencias bibliográficas reales

10.2 Notebook Colab reproducible

- **Archivo**: `UrbanFlow_CDMX_Colab.ipynb` (38 celdas)
- Datos cacheados inline: matriz de transición + 30 rutas de validación
- Corre en Google Colab sin credenciales ni dependencias externas
- Reproduce los resultados de validación reportados en el artículo

10.3 Presentación ejecutiva

- **Archivo**: `UrbanFlow_CDMX_Presentacion.pptx` (16 slides)
- Generada con `pptxgenjs` (Node.js) + iconos de `react-icons`
- Paleta "Midnight Executive" (navy + ámbar)
- Estructura narrativa: portada → problema → metodología → demo → validación → estrategia → cierre

10.4 Documentos complementarios

- `VialAI_Pitch_B2B.pdf` — one-pager ejecutivo de 4 páginas para logística B2B
- `guion_video.md` — guion del video de 5 minutos con timestamps
- `checklist_demo.md` — checklist operativo para la demo en vivo

11 Herramientas, bibliotecas y recursos utilizados

11.1 Lenguajes de programación

- **Python 3.14** — lenguaje principal del motor, agente y pipeline
- **JavaScript (Node.js 22)** — generación de la presentación con `pptxgenjs`
- **LaTeX** — artículo, pitch B2B e inventario
- **Markdown** — guion de video, checklist de demo, README
- **Bash / PowerShell** — scripts de automatización

11.2 Bibliotecas Python (`requirements.txt`)

Biblioteca	Versión	Uso
numpy	1.26+	Álgebra lineal vectorizada
pandas	2.2+	DataFrames
scipy	1.11+	Distribuciones, tests estadísticos
scikit-learn	1.3+	Utilidades de validación
matplotlib	3.8+	Visualizaciones base
seaborn	0.13+	Heatmaps y estilos
geopandas	0.14+	Proyecciones geoespaciales
folium	0.15+	Mapas interactivos

Biblioteca	Versión	Uso
streamlit	1.56+	Dashboard
streamlit-folium	0.18+	Integración mapa-dashboard
plotly	5.18+	Gauges interactivos
pydantic	2.5+	Validación de schemas
anthropic	0.25+	Cliente API Claude
requests	2.31+	HTTP sincrónico
httpx	0.26+	HTTP asíncrono
python-dotenv	1.0+	Carga de variables de entorno
pytest	8.0+	Testing
pytest-cov	4.1+	Cobertura de tests
requests-cache	1.1+	Caché local de respuestas API
`[POR CONFIRMAR]`	—	Otras dependencias del requirements.txt real

11.3 APIs externas consumidas

- **TomTom Traffic Stats API** — https://developer.tomtom.com/traffic-api
- **TomTom Routing API** — https://developer.tomtom.com/routing-api
- **OpenWeatherMap API** — https://openweathermap.org/api
- **Google Maps Distance Matrix API** — https://developers.google.com/maps/documentation/distance-matrix
- **Anthropic Messages API** — https://docs.anthropic.com/en/api/messages
- **CKAN API (datos.cdmx.gob.mx)** — https://datos.cdmx.gob.mx/api/3

11.4 Fuentes de datos públicos

- **C5 CDMX Incidentes Viales** — https://datos.cdmx.gob.mx/dataset/incidentes-viales-c5
- **INEGI Encuesta Origen-Destino ZMVM 2017** — https://www.inegi.org.mx
- **DataMéxico (Sec. Economía)** — https://www.economia.gob.mx/datamexico/es/profile/geo/valle-de-mexico
- **SEMOVI aforos vehiculares** — Gobierno CDMX, datasets abiertos
- **TomTom Traffic Index 2024** — https://www.tomtom.com/traffic-index/mexico-city-traffic/

11.5 Software de desarrollo

- **Visual Studio Code** con extensiones: Python, Pylance, Jupyter, LaTeX Workshop
- **Claude Code** (Anthropic) — asistencia de programación para refactors y debugging
- **Git + GitHub** — control de versiones
- **Python venv** — entorno virtual aislado
- **TeX Live** — distribución LaTeX
- **Node.js 22** — runtime para pptxgenjs
- **LibreOffice** — conversión pptx → pdf para QA visual

11.6 Plataformas de despliegue y distribución

- **GitHub** — `https://github.com/Palermo01/UrbanFlow_CDMX`
- **Google Colab** — ejecución del notebook reproducible
- **Streamlit Community Cloud** / `ngrok` — opciones de demo pública (por confirmar)

11.7 Recursos académicos y bibliográficos

- Norris, J. R. (1997). *Markov Chains*. Cambridge University Press
- Rubinstein & Kroese (2016). *Simulation and the Monte Carlo Method* (3rd ed.). Wiley
- Van Lint & Hoogendoorn (2010). "A Robust and Efficient Method for Fusing Heterogeneous Data from Traffic Sensors." *Computer-Aided Civil and Infrastructure Engineering*, 25(8)
- Harris et al. (2020). "Array programming with NumPy." *Nature*, 585
- Sinnott, R. W. (1984). "Virtues of the Haversine." *Sky and Telescope*, 68(2)

11.8 Recursos de mercado y contexto

- Mordor Intelligence (2025). *Mexico Last Mile Delivery Market Report 2025–2030*
- The Logistics World (2024). "Logística de última milla: innovaciones para entregas rápidas en 2025"
- Asociación Mexicana de Venta Online — *Estudio sobre Venta Online en México 2025*
- T21 (2026). "Última milla, el eslabón más crítico de la logística en México hacia 2030"

12 Resumen ejecutivo del trabajo realizado

12.1 Métricas del proyecto

- **Líneas de código**: `[POR CONFIRMAR]` en `src/` y `app/`
- **Tests automatizados**: 443 tests pasando al 100 %
- **Cobertura de tests**: `[POR CONFIRMAR]` %
- **Fuentes de datos integradas**: 5
- **Perturbaciones en el catálogo**: 11
- **Rutas en el dataset de validación**: 30
- **Páginas del artículo técnico**: 23
- **Slides de la presentación**: 16
- **Celdas del notebook Colab**: 38

12.2 Resultados empíricos

- **MAPE del P50 (VialAI)**: 11.9 %
- **MAPE del ETA (TomTom)**: 19.0 %
- **Ventaja de VialAI sobre TomTom**: 7.1 puntos porcentuales
- **Cobertura empírica de la banda**: 93.3 %
- **MAE en minutos**: 4.8 min
- **Latencia del motor**: <500 ms por consulta

12.3 Entregables finales

1. `UrbanFlow_CDMX_Articulo_v2.pdf` — artículo técnico (23 pp)
2. `UrbanFlow_CDMX_Colab.ipynb` — notebook reproducible
3. `UrbanFlow_CDMX_Presentacion.pptx` — presentación ejecutiva (16 slides)
4. `VialAI_Pitch_B2B.pdf` — pitch comercial para última milla (4 pp)
5. `guion_video.md` — guion del video de 5 minutos
6. `checklist_demo.md` — checklist para demo en vivo
7. `Inventario_Proyecto.pdf` — este documento
8. **Código fuente completo** en https://github.com/Palermo01/UrbanFlow_CDMX

Documento preparado por Carlos Armando López Encino
Diplomado en Ciencia de Datos · FES Acatlán · UNAM
Abril 2026